# Artificial Intelligence

**AIML320/420**

**K-Nearest Neighbour (KNN)**

Dr Fangfang Zhang

SECS & Centre for Data Science and AI

*fangfang.zhang@vuw.ac.nz*

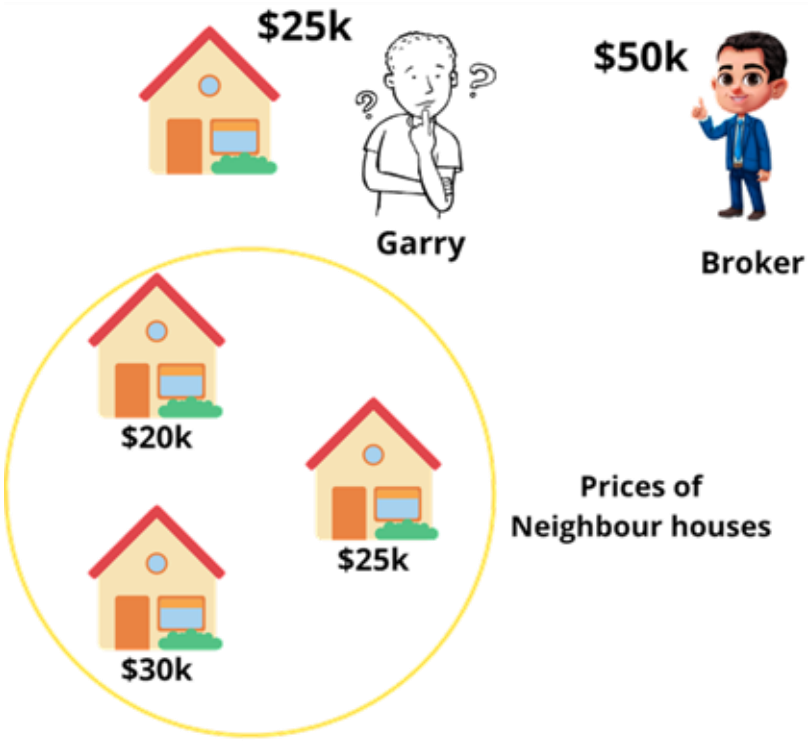
# AIML320/420 Week 2

1. Announcements
   - Class rep: Leon Cai, caileon@myvuw.ac.nz
   - Helpdesk: Tuesday and Thursday, 2-3pm at CO242B
   - Scikit-learn: https://scikit-learn.org/stable/
2. (un)Supervised learning
3. Regression, classification, clustering
4. Dataset split: training, test, and validation set
5. Generalisation and overfitting
6. K nearest neighbour (KNN)

# Outline

- **K-Nearest Neighbour (KNN)**
  - Classification (Supervised Learning)
  - Basic KNN (1-NN)
  - Distance metrics
  - Normalisation
  - Hyperparameter K
  - Classification Evaluation

# K Nearest Neighbours



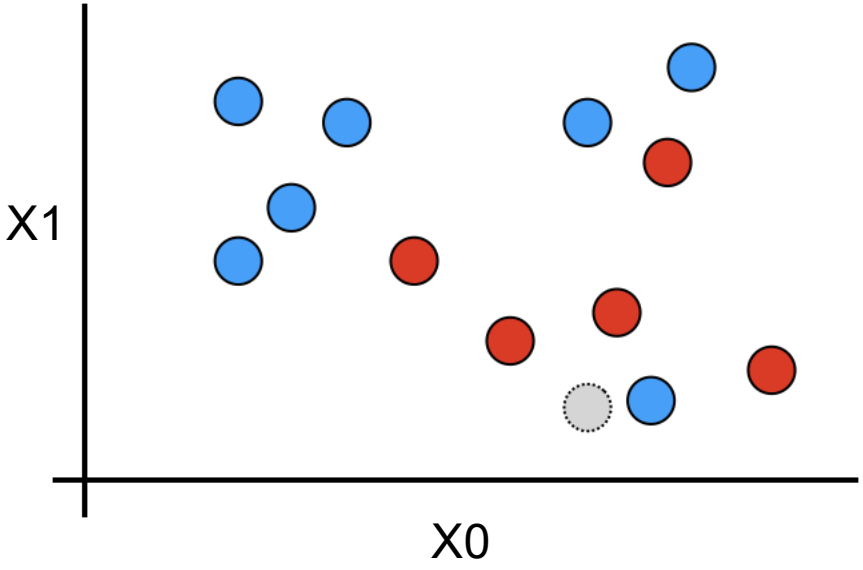
# K Nearest Neighbours

- Taking a new (unseen) instance, a K-nearest neighbours (KNN) classifier:
  - picks the *k* nearest training instances, and outputs the "majority class" within those nearest instances.
  - works out its label according to all training instances.

<u>The basic algorithm</u>

1. <u>Compare/calculate the distances</u> of unseen instance to all training instances;
2. <u>Find the *k* "nearest neighbours"</u> (the training instances that are closest to the unseen instance) from the training set;
3. <u>Classify the unseen instance</u> as according to the nearest neighbours, i.e., majority class.

# 1-NN



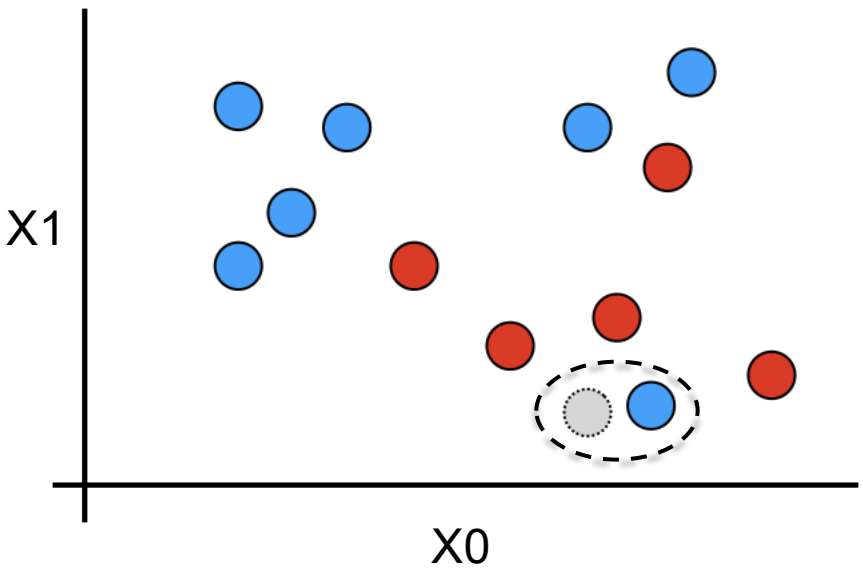

Assume a binary classification problem: blue and red circles

There are two input features X0 and X1

We want to classify the "unknown" gray instance

- **Training:** The training data is shown above (all the red and blue circles)
- We don't know the class for the gray instance.

# 1-NN



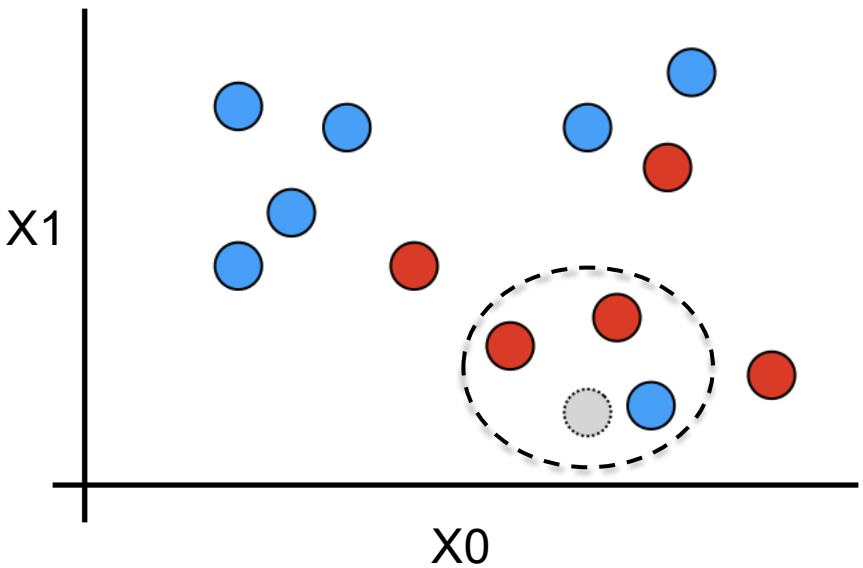

Assume a binary classification problem: blue and red circles

There are two input features X0 and X1

We want to classify the "unknown" gray instance

- **Training:** The training data is shown above (all the red and blue circles)
- We don't know the class for the gray instance.
- **Prediction:** Find the closest neighbor in the training set to the gray inst.
  - The blue instance is the closest neighbor (prediction = blue)

# K-NN




Assume a binary classification problem: blue and red circles

There are two input features X0 and X1

We want to classify the "unknown" gray instance

- **Training:** The training data is shown above (all the red and blue circles)
- We don't know the class for the gray instance.
- **Prediction:** Find the closest neighbor**s** in the training set to the gray inst.
  - Assuming **k = 3**, there are 1 blue and 2 red as nearest neighbors (prediction = red)

# K-NN

- KNN doesn't explicitly learn a model. It just remembers all the training instances.
  - So it's really "fast" to train—actually, there's no training process!
  - "Lazy learning" algorithm
  - But it can be super slow to use/predict, because you have to compute lots of distances each time

# Distance

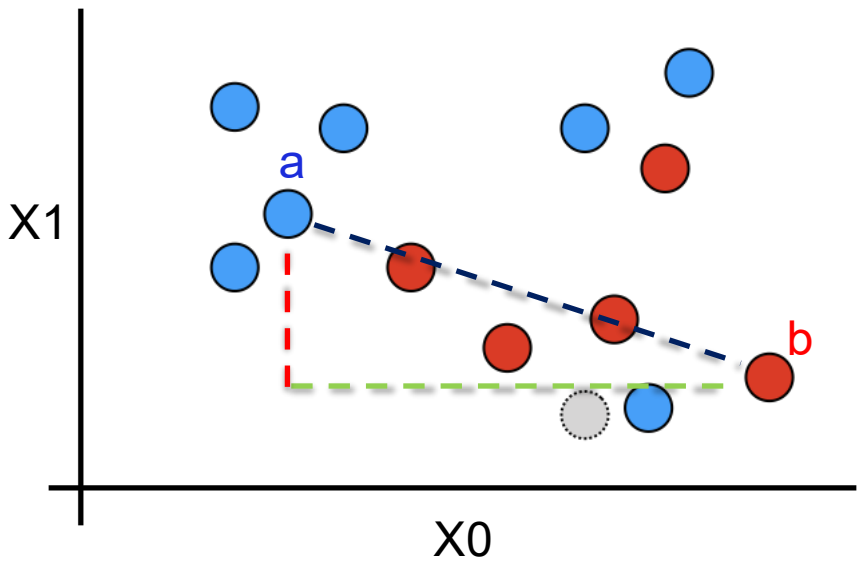
- The notion of “distance” is crucial for KNN (and other ML algorithms)
- Most popular one is the Euclidean distance
  - Given two feature vectors **a** and **b** with **continuous values**

$$d(a,b) = \sqrt{\sum_{i=1}^{n}(a_i - b_i)^2} = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \cdots + (a_n - b_n)^2}$$

Prasatha et al, “Effects of Distance Measure Choice on KNN Classifier Performance - A Review”, https://arxiv.org/pdf/1708.04321

# Euclidean Distance Example

$$d(a,b) = \sqrt{\sum_{i=1}^{n}(a_i - b_i)^2} = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \cdots + (a_n - b_n)^2}$$




$$d(a,b) = \sqrt{(a_{X0} - b_{X0})^2 + (a_{X1} - b_{X1})^2}$$

# Decision Making

- Majority Voting
- Possible Problem:

  *Each of the k nearest neighbours contributes equally*

- *Weighted Distance:*

  **-** Closer neighbours get **higher weights for voting**

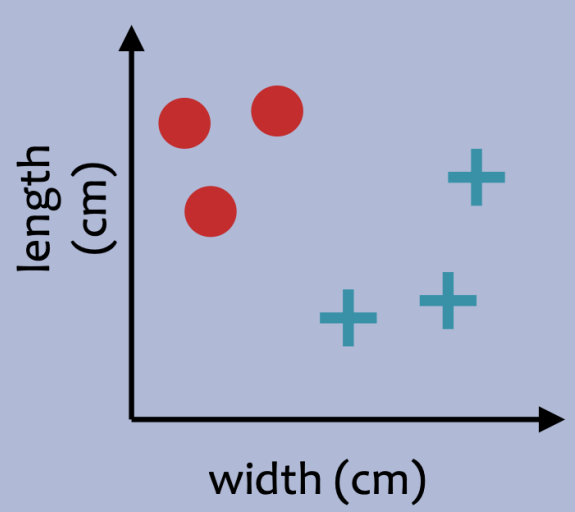
  *weight=1/distance*

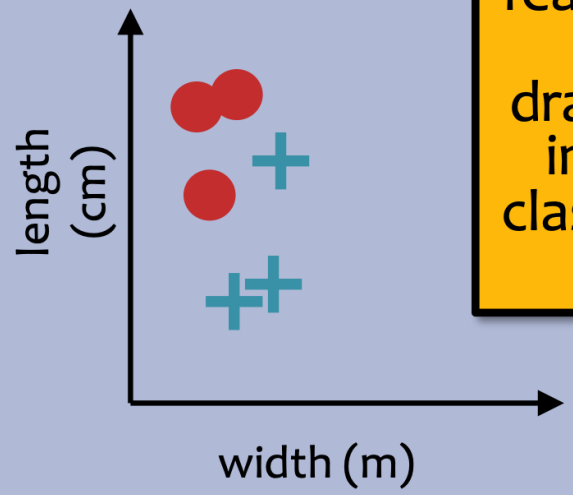
This often leads to **better predictions**, especially when:

- Neighbours are unequally spaced
- Classes are imbalanced or overlapping
- The value of k is large

# Feature Scale Matters

Example: two features for KNN




**big problem:** feature scale could dramatically influence classification results

# Normalization

- We often “normalize” the input data, so that values from all features are within the same range (0,1)
- A common normalization technique is **Min-Max normalization**

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

where x is a feature

- We can apply this directly inside the Euclidean distance calculation:

$$d(a,b) = \sqrt{\sum_{i=1}^{n} \frac{(a_i - b_i)^2}{R_i^2}}$$

assuming $R_i$ as the range for feature $X_i$

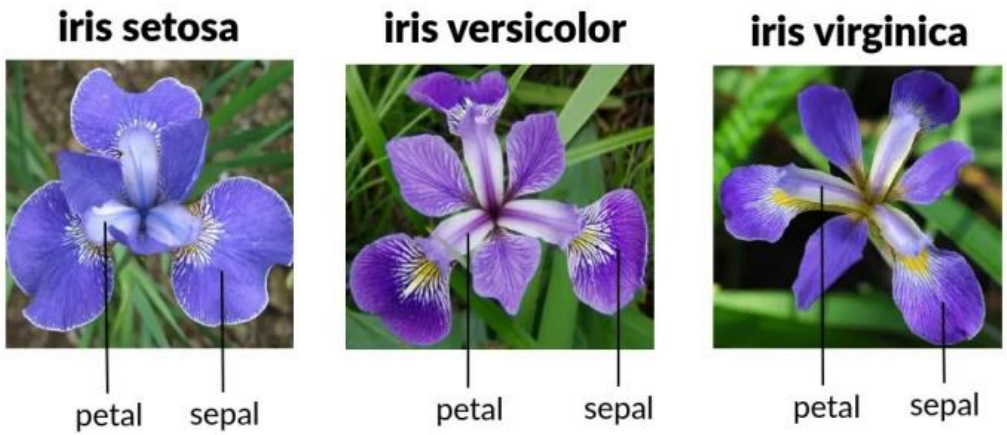
i.e., $x_{max}$ - $x_{min}$

Remember: We only normalize using the training data!

# Different K

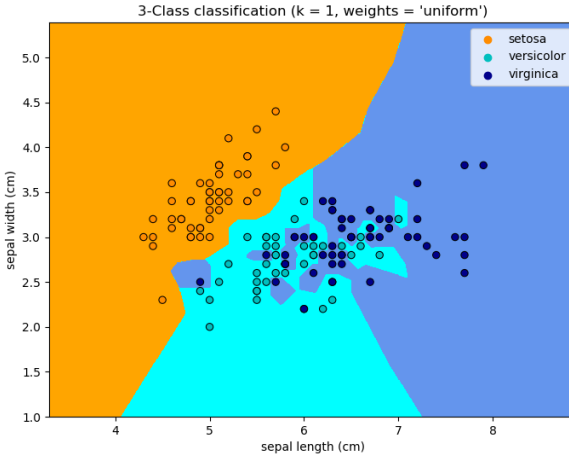
- In simple cases, you can compute the category for each point in the input space. This is called a 'Voronoi tessellation'—useful to illustrate.
- https://en.wikipedia.org/wiki/Voronoi_diagram

- Voronoi diagrams on Iris Dataset to see effects of different K
  - A classic dataset for illustrating ML concept
  - Classifying three types of Iris flower
  - There are four input features for each flower: Petal length, petal width; Sepal length, sepal width

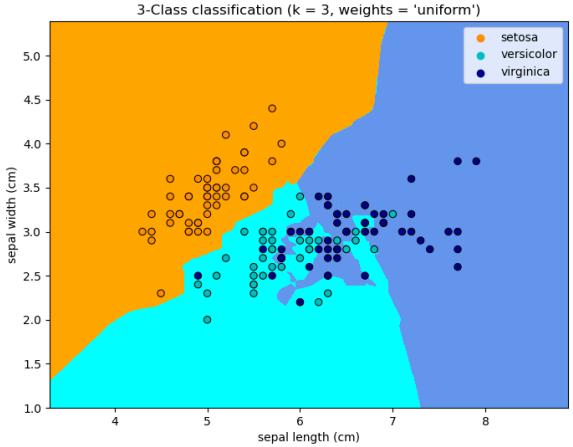


# Voronoi diagrams Visualization

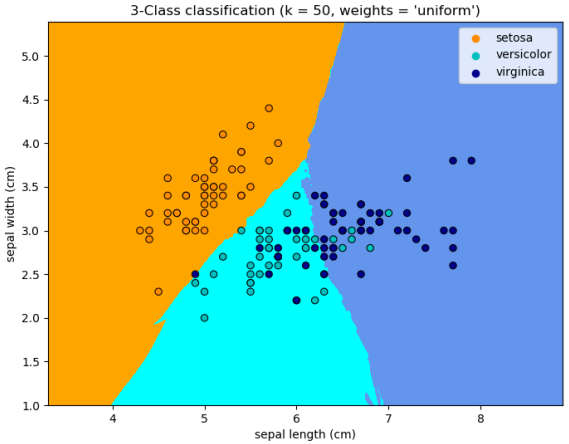
k=1




k=3




k=50




Getting smoother,
but can get slow!

https://scikit-learn.org

# Choosing values for k

- A **small k** focuses on local patterns (can be noisy), while a **larger k** smooths out the noise but may miss local structure.
- How can we find a good value of K
  - We're not allowed to consult the test set, to find a good value of K.
    - The test set must remain unseen, or we're cheating!
  - Instead, we can hold out a second set from training, called the validation set.
    - We can use the validation set to find the right value of K.

# Validation dataset to decide K

- Validation set
  - Separate dataset from the training and test datasets
  - Used for **monitoring the training process** but not directly used for learning the classifier
  - Helps to decide hyperparameter values, in KNN it's K, do **model selection**, and avoid overfitting
- Steps:
  - Split the dataset
  - Define a range of K value to try
  - Train and validate for each k
  - Pick the best k on the validation performance -- How?
  - Use it as the final k for testing

# How Good the Model is?

- To measure the classification performance

| Predicted | Actual Values: Positive (1) | Actual Values: Negative (0) |
|---|---|---|
| Positive (1) | TP | FP |
| Negative (0) | FN | TN |

Confusion Matrix

TP: True Positive; FP: False Positive;
FN: False Negative; TN: True Negative

There are several Metrics: Accuracy, Precision, Recall, F1-score

$$\textbf{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Overall classification accuracy

$$\textbf{Recall} = \frac{TP}{TP + FN} \text{ or } \frac{\text{True Positive}}{\text{Actual Results}}$$

For the positive instances, how many of them are correctly predicted?

$$\textbf{Precision} = \frac{TP}{TP + FP} \text{ or } \frac{\text{True Positive}}{\text{Predictive Results}}$$

Among predicted positive instances, how many of them are really positive?

$$\textbf{F - score} = \frac{2 * \text{Recall} * \text{Precision}}{\text{Recall} + \text{Precision}}$$

# Example of SPAM Detection

- Amongst the 200 emails, 80 emails are actually spam in which the model correctly identifies **60** of them as spam (TP).
- Amongst the 200 emails, 120 emails are not spam in which the model correctly identifies **100** of them as not spam (TN).

| Predicted/Actual | Spam (Positive) | Not spam (Negative) |
|---|---|---|
| Spam (Positive) | 60 (TP) | 20 (FP) |
| Not spam (Negative) | 20 (FN) | 100 (TN) |

Confusion Matrix

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} = \frac{60+100}{60+100+20+20} = \frac{160}{200} = 0.8$$

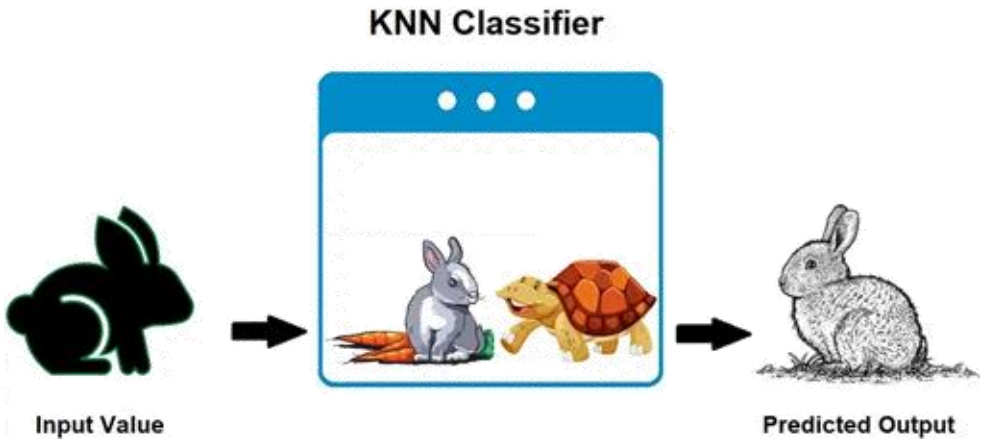
$$Precision = \frac{TP}{TP+FP} = \frac{60}{60+20} = \frac{60}{80} = 0.75$$

$$Recall = \frac{TP}{TP+FN} = \frac{60}{60+20} = \frac{60}{80} = 0.75$$

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} = 2 \times \frac{0.75 \times 0.75}{0.75 + 0.75} = 0.75$$

# Limitations of KNN

- Curse of dimensionality, as the number of features (dimensions) increases, the "distance" between points becomes less meaningful.
- High computational cost, i.e., the algorithm must calculate the distance to *every* point in the dataset.
- Sensitivity to outliers and noise, relies on local neighborhoods, "dirty" data points or outliers can easily pull the prediction in the wrong direction.
- Imbalanced data bias: If one class appears much more frequently than another, it will naturally dominate the "neighborhood" of any new point.
- The prediction is limited to the samples in KNN.



# Some thoughts

1. *Can we use KNN for regression?*

   Yep, we can use the “value” from neighbors as well.

2. *Prefer to use an odd or even number as k in KNN?*

   odd, majority voting, easier to make decisions;

   if even, can randomly select one.

# Some thoughts

*3. Should we normalize test data when using KNN?*

Sure! Need to consider the scales of features

*4. How to normalize test data in KNN? Min and Max values from where? Training or test?*

use the feature value ranges from training

each instance in test is independent, no overall information of whole test instances (if use, also cause “Data Leakage”)

the training model is from training, and better to keep the same

# Summary

- **KNN** is a simple yet powerful **supervised learning** method
- Fast to train, no train actually
- Can be slow to test new instance
- It is crucial to choose the **evaluation metric** appropriately, either for classification or regression
- Example codes for KNN is available on course website

Next Lecture:

- Decision Tree